

Frontend Architecture and File Structure

The frontend is a React 18, TypeScript, Vite, and Tailwind CSS single-page application.

Folder structure:-

```
frontend/  
├── src/  
│   ├── api/  
│   ├── auth/  
│   ├── components/  
│   │   ├── chat/  
│   │   ├── common/  
│   │   ├── editor/  
│   │   ├── project/  
│   │   └── workspace/  
│   ├── contexts/  
│   ├── hooks/  
│   ├── layouts/  
│   ├── pages/  
│   ├── routes/  
│   ├── stores/  
│   ├── types/  
│   ├── utils/  
│   ├── App.tsx  
│   ├── main.tsx  
│   └── index.css  
├── Dockerfile  
├── nginx.conf  
├── vite.config.ts  
├── tailwind.config.js  
├── postcss.config.js  
├── tsconfig.json  
├── package.json  
└── index.html
```

1. Application entry files

src/

├─ main.tsx

├─ App.tsx

└─ index.css

main.tsx

The browser entry point.

Responsibilities:

- Finds the HTML root element
- Mounts the React application
- Enables React Strict Mode
- Imports global CSS
- Starts the component tree

index.html

↓

main.tsx

↓

App.tsx

App.tsx

The root application component.

Responsibilities:

- Initializes authentication state
- Mounts global providers
- Creates the browser router
- Renders application routes

Conceptually:

AppProviders

↓

BrowserRouter

↓

AppRoutes

index.css

Contains:

- Tailwind directives
- Global typography
- Root colours
- Application-wide styles
- Reusable CSS utilities

2. API layer

src/api/

```
├— client.ts
├— auth.ts
├— projects.ts
├— files.ts
├— chat.ts
└— workspaces.ts
```

The API layer isolates all communication with the backend.

Components and pages should not construct URLs or call Axios directly.

client.ts

Central Axios client.

Responsibilities:

- Configures the /api/v1 base URL
- Adds JWT access tokens
- Enables refresh-token cookies
- Handles 401 Unauthorized
- Refreshes expired sessions
- Normalizes HTTP configuration

auth.ts

Authentication requests:

- Signup

- Login
- Refresh token
- Logout
- Fetch current user

[projects.ts](#)

Project operations:

- List projects
- Create project
- Import local project
- Update project
- Delete project
- Fetch individual project

[files.ts](#)

File-system operations:

- Fetch project tree
- Fetch file contents
- Create files
- Update files
- Delete files
- Create folders
- Delete folders

[chat.ts](#)

AI communication:

- Fetch chat history
- Send non-streaming chat requests
- Establish WebSocket chat streams
- Receive AI token deltas
- Undo AI modifications

[workspaces.ts](#)

Retrieves available workspace definitions:

- JavaScript
- Python
- Website Builder

3. Authentication layer

src/auth/

└─ ProtectedRoute.tsx

ProtectedRoute.tsx

Protects private pages.

It checks authentication state before rendering the requested route.

Requested protected route

↓

Authentication bootstrapping?

└─ Yes → Show loading spinner

└─ No

↓

Authenticated?

└─ Yes → Render page

└─ No → Redirect to login

Public-only route behavior also prevents authenticated users from returning to login or signup pages.

4. Reusable components

src/components/

└─ chat/

└─ common/

└─ editor/

└─ project/

└─ workspace/

Components are grouped by feature rather than placed in one large directory.

4.1 Common components

components/common/

└─ Button.tsx

└─ Input.tsx

└─ Modal.tsx

These components form the basic design system.

They provide consistent:

- Button variants
- Form inputs
- Validation modals
- Loading spinners
- Empty states
- Colours and spacing
- Disabled and focus states

They should not contain project-specific business logic.

4.2 Editor components

components/editor/

```
├─ CodeEditor.tsx
├─ EditorTabs.tsx
├─ FileExplorer.tsx
└─ LivePreview.tsx
```

CodeEditor.tsx

Wraps Monaco Editor.

Responsibilities:

- Syntax highlighting
- Language selection
- Editing file content
- Read-only mode
- Saving status
- Detecting blocked editing attempts
- Monaco configuration

EditorTabs.tsx

Displays:

- Open file tabs
- Active file state
- Close controls
- Website preview tab

The actual tab state is stored separately in editorStore.ts.

FileExplorer.tsx

Renders the project file tree recursively.

Responsibilities:

- Display files and folders
- Highlight the active file
- Open files
- Create files and folders
- Delete files and folders

LivePreview.tsx

Runs Website Builder projects inside a sandboxed iframe.

It supports two preview modes.

Static preview

index.html

+

CSS files

+

JavaScript files

↓

Combined preview document

React preview

React entry file

↓

esbuild-wasm

↓

Bundled JavaScript

↓

Sandboxed iframe

It also handles:

- TypeScript and TSX compilation
- Package loading through esm.sh
- Tailwind preview support
- Runtime error detection
- Missing-module diagnostics
- Virtual React entry generation

- Automatic preview repair
- Opening preview in a new tab

4.3 Chat components

components/chat/

└─ ChatPanel.tsx

ChatPanel provides the AI assistant interface.

Responsibilities:

- Render chat messages
- Display streamed AI responses
- Accept user prompts
- Disable input during generation
- Cancel active generation
- Display applied file-change counts
- Undo the latest AI changes
- Hide structured file blocks from conversational output

It is a presentation component. Chat networking and orchestration remain in the API layer and useProjectEditor.

4.4 Project components

components/project/

└─ ProjectCard.tsx

Displays project information such as:

- Project name
- Description
- Workspace type
- Last update time
- Open action
- Delete action

4.5 Workspace components

components/workspace/

└─ WorkspaceCard.tsx

Displays one workspace category:

- JavaScript Workspace
- Python Workspace
- Website Builder

It navigates users to the corresponding project dashboard.

5. Context providers

src/contexts/

└─ AppProviders.tsx

AppProviders contains application-wide providers.

Currently, its main responsibility is configuring TanStack React Query.

React Query manages server state including:

- Projects
- Project metadata
- File trees
- Chat history
- Workspace definitions

Default query behavior includes:

- Limited retries
- Disabled automatic refetch on window focus
- Central query caching

6. Custom hooks

src/hooks/

└─ useProjectEditor.ts

useProjectEditor is the primary orchestration layer for the editor page.

It coordinates:

- Project loading
- File-tree loading
- Chat history
- Opening and closing files
- Editor tab selection
- Autosaving
- File and folder creation
- File deletion
- AI WebSocket communication
- Optimistic chat messages
- AI file-change synchronization
- Undo
- Website preview
- Preview repair
- Project ZIP download
- Workspace editing restrictions

Conceptually:

EditorPage

↓

useProjectEditor

- └─ React Query
- └─ Zustand stores
- └─ File APIs
- └─ Chat WebSocket
- └─ Preview state
- └─ Editor actions

This keeps EditorPage.tsx focused on layout and component composition.

7. Layouts

src/layouts/

- └─ AppLayout.tsx
- └─ AuthLayout.tsx

[AppLayout.tsx](#)

Used by authenticated pages.

Provides:

- Application header
- User information
- Logout controls
- Main content container
- Shared authenticated navigation

[AuthLayout.tsx](#)

Used by login and signup pages.

Provides a consistent authentication screen structure without the main application navigation.

8. Pages

src/pages/

```
├— LoginPage.tsx
├— SignupPage.tsx
├— WorkspacesPage.tsx
├— ProjectDashboardPage.tsx
└— EditorPage.tsx
```

Pages are route-level components.

[LoginPage.tsx](#)

- Login form
- Form validation
- Authentication request
- Error handling
- Redirect after success

[SignupPage.tsx](#)

- Registration form
- Input validation
- Account creation
- Session initialization

[WorkspacesPage.tsx](#)

- Loads available workspace types
- Displays workspace cards

- Acts as the main application entry after login

ProjectDashboardPage.tsx

- Lists projects for a selected workspace
- Creates projects
- Imports local folders
- Deletes projects
- Opens projects in the editor

EditorPage.tsx

Composes the main development interface:

EditorPage

└─ Header

└─ FileExplorer

└─ EditorTabs

└─ CodeEditor or LivePreview

└─ ChatPanel

└─ ValidationModal

EditorPage does not directly implement the entire editor workflow. It delegates orchestration to useProjectEditor.

9. Routing

src/routes/

└─ AppRoutes.tsx

The route structure is:

/login

/signup

/workspaces

/workspaces/:workspaceType

/projects/:projectId

Protected routes require an authenticated user.

AppRoutes

└─ Public routes

```
|   ├── /login
|   └── /signup
└── Protected routes
    ├── /workspaces
    ├── /workspaces/:workspaceType
    └── /projects/:projectId
```

Unknown routes redirect to the workspace page.

10. State management

src/stores/

```
├── authStore.ts
├── editorStore.ts
└── uiStore.ts
```

The frontend distinguishes between server state and client state.

Server state

Managed with TanStack React Query:

- Projects
- Files
- File trees
- Chats
- Workspace information

Client state

Managed with Zustand and component state.

authStore.ts

Stores:

- Current user
- Authentication status
- Access token
- Bootstrap status

The access token is kept in memory. The refresh token remains in an HTTP-only cookie.

editorStore.ts

Stores:

- Open editor tabs
- Active tab
- File contents in open tabs
- Dirty status
- Saved content state

uiStore.ts

Stores interface-level preferences such as:

- Whether the AI chat panel is open

Temporary states such as saving, streaming content, and preview compilation remain inside useProjectEditor.

11. Shared types

src/types/
└─ index.ts

Contains TypeScript contracts for:

- Users
- Workspaces
- Projects
- Files
- Folders
- File-tree nodes
- Editor tabs
- Chat messages
- File-change proposals
- API responses

These types keep APIs, stores, hooks, pages, and components consistent.

12. Utility modules

src/utls/
└─ helpers.ts

```
├— workspaceFilePolicy.ts
├— localFolderImport.ts
└— downloadProjectZip.ts
```

helpers.ts

General reusable functions such as:

- Class-name composition
- Error-message extraction
- Date or display helpers

workspaceFilePolicy.ts

Defines frontend editing rules.

Examples:

- Python workspace: .py
- JavaScript workspace: .js, .jsx, .ts, .tsx
- Website workspace: unrestricted frontend project files

The backend repeats these checks because frontend validation alone is not a security boundary.

localFolderImport.ts

Reads a local folder selected by the user.

It:

- Traverses selected files
- Ignores node_modules
- Rejects unsupported binary files
- Limits imported file count
- Builds the project import payload

downloadProjectZip.ts

Downloads the current project as a compressed ZIP archive.

13. Build and deployment files

frontend/

```
├— Dockerfile
├— nginx.conf
└— vite.config.ts
```

```
├─ tailwind.config.js
├─ postcss.config.js
├─ tsconfig.json
├─ package.json
└─ index.html
```

[vite.config.ts](#)

Configures:

- React plugin
- @/ source alias
- Development API proxy
- WebSocket proxy
- Build behavior

[nginx.conf](#)

Production responsibilities:

- Serve compiled frontend files
- Support SPA route fallback
- Proxy /api/ to FastAPI
- Proxy /ws/ WebSockets
- Configure long WebSocket timeouts

[Dockerfile](#)

Uses a multi-stage build:

Node build stage

↓

Vite production build

↓

Nginx runtime image

[tailwind.config.js](#)

Defines Tailwind source paths, theme extensions, typography, and colours.

[tsconfig.json](#)

Defines:

- TypeScript compiler settings
- Strictness

- JSX support
- Path aliases

14. Frontend dependency direction

The recommended dependency flow is:

Pages

↓

Feature Components

↓

Hooks and Stores

↓

API Modules

↓

Backend

Shared modules support every layer:

Common Components

Types

Utilities

Contexts